МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Андрианов Антон
Группа: М8О-312БВ-24
Преподаватель: Шангин Даниил

Москва, 2026

Условие

Цель работы — освоить построение трёхмерных объектов, перспективную и ортографическую проекции, аффинные преобразования и отрисовку средствами Vulkan с интерфейсом GLFW и ImGui.

Выполнен вариант 2: правильный октаэдр. Программа отображает фигуру в реальном времени и позволяет изменять её параметры через интерфейс.

Реализованы все дополнительные задания: переключение проекции; изменение позиции, поворота и масштаба; движение и вращение по сложной траектории с паузой и настройкой параметров; изменение цвета; процедурные цвета вершин с умножением на выбранный цвет; отображение нескольких объектов с отдельными наборами VkDescriptorSet.

Метод решения

Геометрия октаэдра задана шестью вершинами: (1, 0, 0), (−1, 0, 0), (0, 1, 0), (0, −1, 0), (0, 0, 1), (0, 0, −1). Массив из 24 индексов задаёт восемь треугольных граней. Длина каждого из 12 рёбер равна $\sqrt{2}$, поэтому исходная фигура является правильным октаэдром.

Математика реализована в source/application.cpp. Функции translation(), rotation() и scaling() строят матрицы перемещения, поворота и масштабирования. В model_matrix() они объединяются в матрицу модели $M = T \cdot Rz \cdot Ry \cdot Rx \cdot S$. Для вектора-столбца преобразования выполняются справа налево: сначала масштаб, затем повороты вокруг X, Y и Z, затем перемещение. Углы из интерфейса переводятся из градусов в радианы.

Функция projection() строит перспективную или ортографическую матрицу P. Матрица вида V задаётся перемещением на −camera_distance по

оси Z. В render() вычисляется MVP = P · V · M. Учитывается соотношение сторон области сцены, диапазон глубины Vulkan [0, 1] и направление оси Y. Пользователь переключает проекцию в разделе Projection и задаёт угол обзора либо размер ортографической области.

В разделе Objects доступны Position, Rotation и Scale для изменения параметров по трём осям. Анимация первого объекта рассчитывается по формулам x = x + r·sin(t), y = y + h·sin(2t), z = z + 0,5r·sin(3t). Углы поворота дополнительно увеличиваются на (25t, 50t, 15t) градусов. Время t накапливается с учётом интервала между кадрами и параметра Speed. Play включает движение или ставит его на паузу, Radius и Height меняют траекторию, Restart сбрасывает фазу.

Процедурный цвет вершины вычисляется из её локальной позиции: c = (position + 1) / 2. Переключатель Vertex colors включает этот режим. В вершинном шейдере цвет вершины умножается на tint — цвет, выбранный через ImGui::ColorEdit3. При отключении Vertex colors используется только tint. Между вершинами цвет интерполируется.

В initialize() создаются vertex buffer, index buffer, descriptor set layout, descriptor pool, pipeline layout и graphics pipeline. Для каждого из трёх объектов выделяются отдельный uniform buffer и VkDescriptorSet. Uniform buffer содержит матрицы MVP и M, цвет tint и параметр использования цветов вершин. Буферы выделяются через Vulkan Memory Allocator.

Каждый кадр main.cpp обрабатывает события GLFW, формирует интерфейс ImGui и вызывает update(), prepare(), render() и submitAndPresent(). prepare() ожидает завершения предыдущего кадра. Затем render() обновляет uniform buffers, записывает команды в command buffer, очищает цвет и глубину, подключает pipeline и буферы геометрии. Для каждого видимого объекта привязывается его descriptor set и

вызывается vkCmdDrawIndexed().

Вершинный шейдер shaders/octahedron.vert вычисляет gl_Position = MVP · vec4(position, 1). Видеокарта растеризует треугольники, а фрагментный шейдер shaders/octahedron.frag рассчитывает цвет с простым направленным освещением. Проверка глубины скрывает поверхности, расположенные за другими гранями. Готовое изображение выводится через swapchain.

Режим Multiple objects отображает три октаэдра. Selected выбирает объект для редактирования, Visible управляет его видимостью. Общие буферы геометрии используются повторно, а отдельные uniform buffers и descriptor sets обеспечивают независимые трансформации и цвета объектов.

Результаты

Программа собрана в Ubuntu с помощью CMake и GCC. Для совместимости с vk-bootstrap используются заголовки Vulkan 1.4.321; подключается системная библиотека Vulkan. Сборка завершилась успешно. Автоматический тест octahedron-math прошёл: 1 из 1.

Тест проверяет геометрию октаэдра, матрицы проекций и преобразований, а также поведение времени анимации при паузе и изменении скорости. Эти проверки дополняют демонстрацию работы интерфейса.

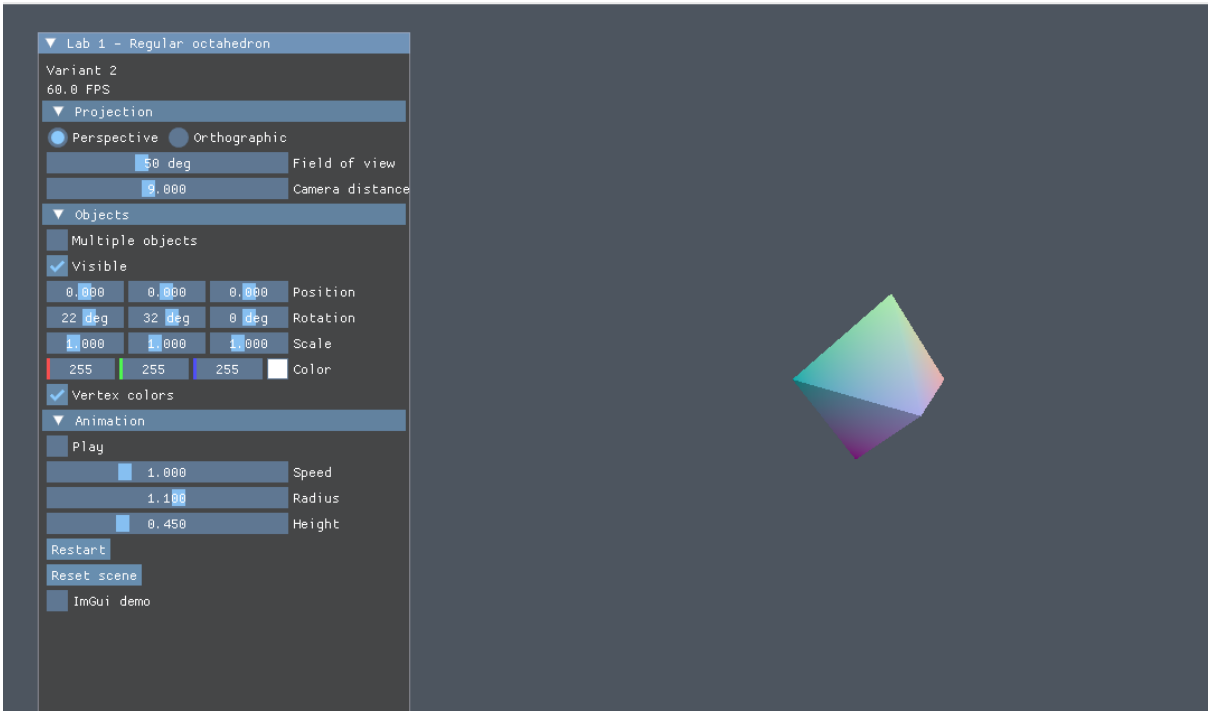


Рисунок 1 — Правильный октаэдр с процедурными цветами вершин в перспективной проекции.

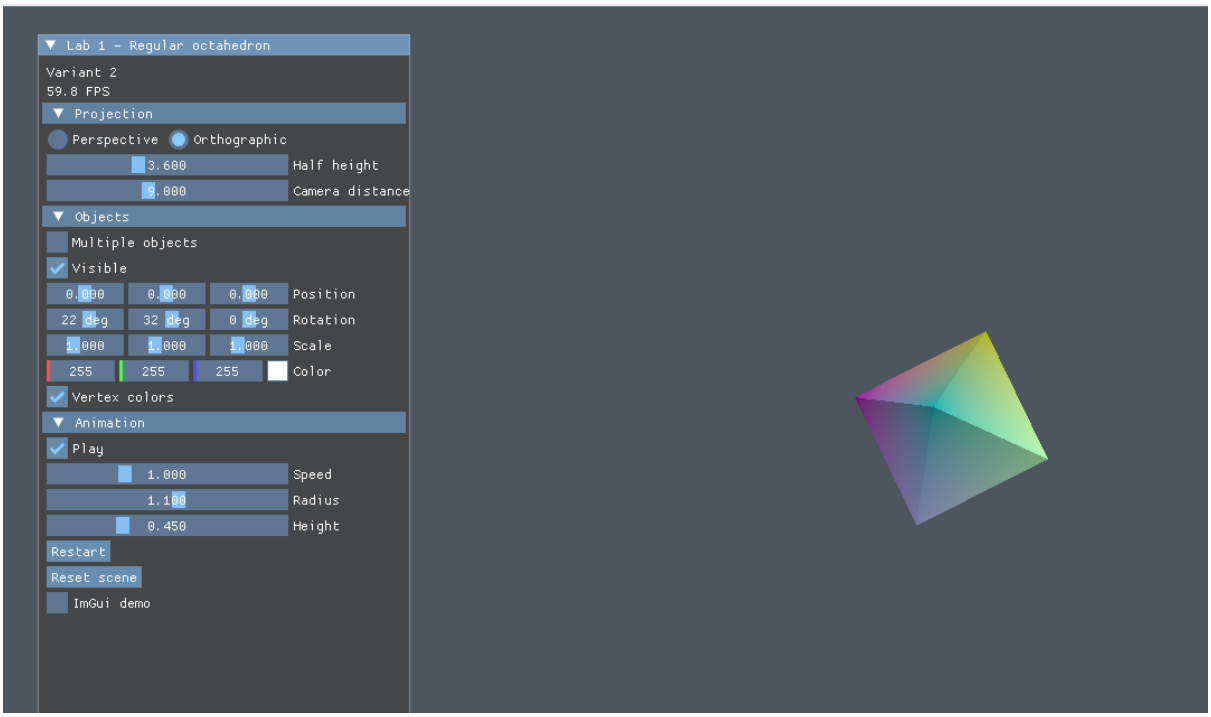


Рисунок 2 — Ортографическая проекция и управление анимацией: скорость, радиус и высота траектории.

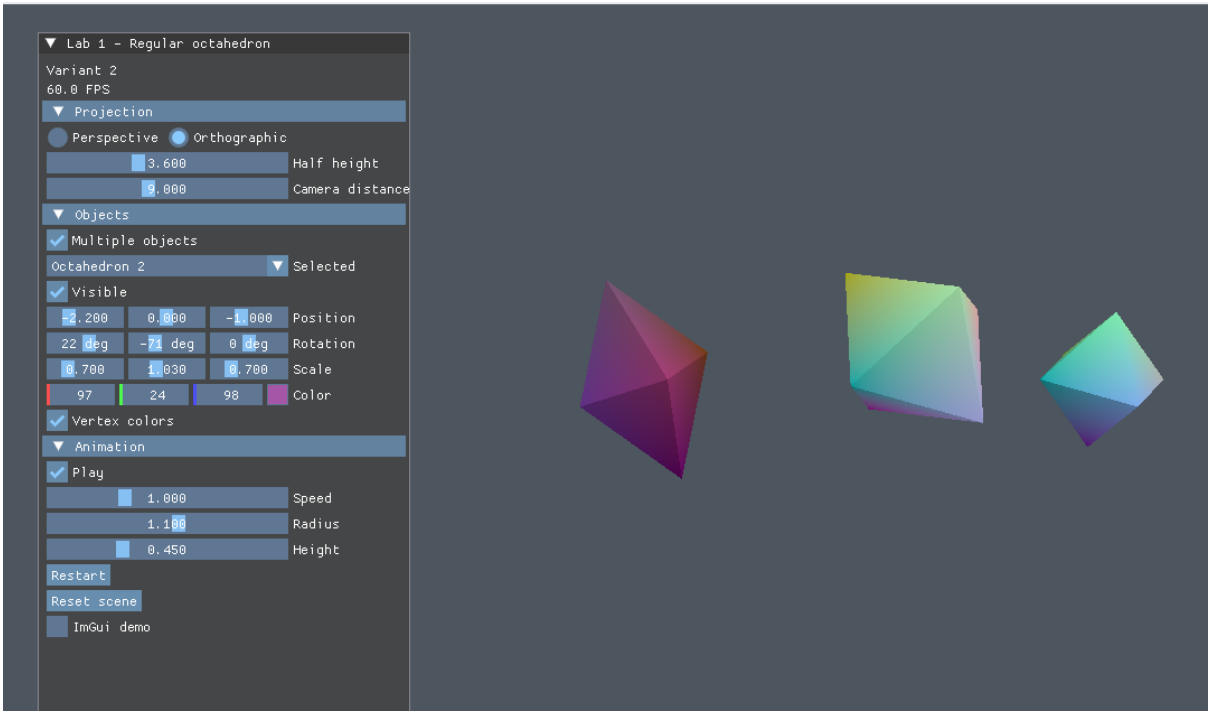


Рисунок 3 — Три объекта с независимыми трансформациями и цветами, использующие отдельные VkDescriptorSet.

Выводы

В ходе лабораторной работы я научился задавать трёхмерную фигуру вершинами и индексами, строить матрицы перемещения, поворота, масштабирования и проекции, а также передавать данные шейдерам через uniform buffers и descriptor sets.

Я реализовал управление фигурой через ImGui, движение по сложной траектории, процедурную окраску и отрисовку нескольких объектов. Работа помогла понять последовательность формирования кадра: расчёты на CPU, запись команд Vulkan, выполнение шейдеров на GPU и вывод готового изображения. Выполнен вариант 2 и все шесть дополнительных заданий.